

component layout in Fig. 15. Note that the PCB has a provision for connecting an LED, though it is not used in the thermometer program.

You may visit the 'www.ti.com/msp430' website to get access to all the datasheets, user guides, application notes, example codes and development

tools pertaining to the MSP430 chip.

EFY note. All the relevant source codes of this article have been included in this month's EFY-CD.

BLINKING_LED.C

```
//Appendix C Software for blinking LED
//This is a program to create a blinking
//led and switch control to put
//the system to sleep mode/active mode.
#include <msp430x20x3.h>
/*
Main routine begins here
*/
long int delVal; // to store delay
value
long int i;
short int sysOn; // variable to
store the state of the system
// sysOn = 1 ==> system in active
// sysOn = 0 ==> system in sleep
mode
void main( void)
{
WDCTCL = WDTN + WDTTHOLD;
// Stop WDT
sysOn=0x01; // system in Active mode
* setup BCM (clock module)
* By default MCLK and SMCLK will be on
internal DCO at 1 Mhz approx.
* Program ACLK on VLO at 10 kHz approx.
*/
BCSCTL3 = LFX1T1S;

```

```
/* Setup P1 ports as outputs and out-
put zero
* Setup P2.6 as output to drive led and
p2.7 as input
* to detect interrupt through switch */
P1DIR = 0xFF;
P1OUT = 0x00;
P2SEL = 0x00; // configure P2.6 and
P2.7 as digital I/O
P2DIR = 0x40;
P2OUT = 0x40;
P2IE = 0x80; // p2.7 is interrupt
sensitive
P2IES = 0x80; // interrupt occurs on
falling edge
_EINT(); // enable global in-
terrupts
/* Blinking led with a delay routine */
f o r // infinite loop
{
while (sysOn == 1)
{
for (delVal = 50000; delVal > 0; del-
Val--);
P2OUT ^= 0x40; // toggle LED with a
delay

```

```
} // program comes here after first
switch interrupt service
P 2 O U T = 0 x 0 0 ;
// switch off LED and enter sleep mode
_BIS_SR(LPMA bits); // enter sleep
mode if sysOn == 0
- N O P ( ) ;
// Required only for C-spy
// program comes here after second
switch interrupt service
// and re-enters the loop
}
/* PORT2 ISR is entered whenever switch
is pressed
exit out of lpm mode */
#pragma vector=PORT2_VECTOR
interrupt void port2ISR( void)
{
sysOn=0x01; //toggle sysOn state
for(i=1000;i>0;i--); //debounce delay
P2IFG &= 0x80; //reset the interrupt
flag
_low_power_mode_off_on_exit();
}

```

DIGIT_THERMO.C

```
/* Program to read out temperature from
a thermistor directly on a
digital multi meter */
#include <msp430x20x3.h>
float varSupply; // store
power supply value
unsigned int varResT; // ther-
mistor resistance by 10
unsigned long int refRes;
// reference resistor
unsigned int tempDegC;
// temperature value
unsigned int WADC;
// Storing ADC result
unsigned int pwmHigh;
// pwm Duty cycle
unsigned int pwmPeriod;
// pwm period
long int i;
short int sysOn; // variable to
store the state of the system
// sysOn = 1
==> system in active
// sysOn = 0
==> system in sleep mode
// function declaration
unsigned int getTempDegC(unsigned int
resVal);
// constants defined
#define MAXTEMPNUM 99 // number of
entries in lookup table
#define REFRESVAL 68000 // value of
ref resistor/10
/* Thermistor resistance to temperature
conversion table

```

```
Resistance is divided by 10 while
temperature is multiplied
by 10 to ensure that the table values
fit as 16-bit integers
*/
const unsigned int res_temp_
lookup[MAXTEMPNUM][2] {
{16700, 0}, {16200, 10}, {15700,
20}, {15200, 30}, {14700, 40}, {14200,
51}, {13700, 62}, {13200, 74}, {12700,
86}, {12200, 99}, {11900, 107}, {11600,
115}, {11300, 123}, {11000, 132}, {10700,
141}, {10400, 150}, {10100, 159}, {9800,
169}, {9500, 179}, {9200, 189}, {8700,
207}, {8500, 214}, {8300, 222}, {8100,
230}, {7900, 238}, {7700, 246}, {7500,
254}, {7300, 263}, {7100, 272}, {6900,
281}, {6700, 291}, {6500, 300}, {6300,
311}, {6100, 321}, {5900, 332}, {5700,
345}, {5500, 354}, {5300, 366}, {5100,
379}, {5000, 385}, {4900, 392}, {4800,
399}, {4600, 412}, {4500, 419}, {4300,
434}, {4200, 442}, {4100, 450}, {3900,
465}, {3800, 474}, {3700, 483}, {3600,
492}, {3500, 501}, {3400, 510}, {3300,
520}, {3200, 530}, {3100, 540}, {3000,
551}, {2900, 562}, {2800, 573}, {2700,
585}, {2600, 597}, {2500, 610}, {2400,
617}, {2380, 626}, {2320, 634}, {2260,
642}, {2200, 651}, {2140, 660}, {2080,
669}, {1960, 688}, {1900, 698}, {1840,
708}, {1780, 719}, {1720, 730}, {1660,
745}, {1600, 753}, {1560, 761}, {1520,
769}, {1440, 786}, {1400, 795}, {1360,
804}, {1320, 813}, {1280, 823}, {1240,
833}, {1200, 843}, {1160, 854}, {1120,
865}, {1080, 876}, {1040, 888}, {1020,
894}, {1000, 900}, {980, 906}, {940,
919}, {900, 932}, {880, 939}, {840,
954}, {780, 976}, {740, 992}, {720, 1000}

```

```
};
/*
Function that fetches the temperature
for a given resistance value
This function does linear interpolation
if the resistance values
lies between two adjacent table en-
tries. This ensures high accuracy
of measurement
*/
unsigned int getTempDegC(unsigned int
resVal)
{
unsigned int tempVal;
unsigned int i;
unsigned int t2;
unsigned int t1;
unsigned int r2;
unsigned int r1;
if (resVal >= res_temp_lookup[0][0])
{
tempVal = 0; // temperature is <=
zero degree celsius
} else if (resVal <= res_temp_
lookup[MAXTEMPNUM-1][0])
{
tempVal = 100; // temperature is
>= 100°C
} else
{
for (i = 0; i<102; i++)
{
if (resVal > res_temp_
lookup[i][0])
{
t1 = res_temp_lookup[i-1][1];
t2 = res_temp_lookup[i][1];
r1 = res_temp_lookup[i-1][0];

```